Real-Time Streaming Analytics

A Student Guide to Building a Mini-Project

How to design, build, and present a live data-streaming pipeline with machine learning

Course: Data Stream Analytics
Worked Example: Smart-Meter Power Consumption

Kafka • Python • scikit-learn • MySQL • Streamlit

Contents

How to Use This Guide 1

1. What Is Real-Time Streaming Analytics? 1

2. What Your Mini-Project Must Contain 1

3. The Worked Example: Smart-Meter Data 1

4. Layers 1 & 2: Streaming the Data with Kafka 1

4.1 The producer 1

5. Layer 3: The Stream Processor (Windowing) 1

6. Layer 4: The Machine-Learning Model 1

6.1 The train / serve split 1

6.2 Training the model 1

6.3 Scoring live in the processor 1

7. Layer 5: The Database (Serving Layer) 1

8. Layer 6: The Live Dashboard 1

9. Running the Whole Pipeline 1

10. Demonstrating It Live 1

11. Writing Your Own Report 1

12. Ideas to Make Yours Stand Out 1

Appendix: Submission Checklist 1

How to Use This Guide

This guide shows you how to build a complete real-time streaming analytics mini-project, from raw data to a live dashboard with machine learning. It uses one worked example — household smart-meter data — but **you are free to choose a different dataset and different tools**. What matters is that your project follows the same six-layer architecture and meets the requirements in Section 2.

Read Section 1 to understand the big picture, Section 2 for what you must submit, and Sections 3–9 for the step-by-step build. The final sections cover the live demo and how to write your own report.

1. What Is Real-Time Streaming Analytics?

A **batch** system collects data first and analyses it later — like downloading a file and opening it in Excel. A **streaming** system analyses data *as it arrives*, continuously, so results are always current. Think of a live heart-rate monitor versus a report printed at the end of the day.

Every streaming project, whatever the topic, is built from the same six layers. Data is produced, transported, processed, optionally scored by a model, stored, and shown on a live view. Learn these six layers once and you can build any streaming project.

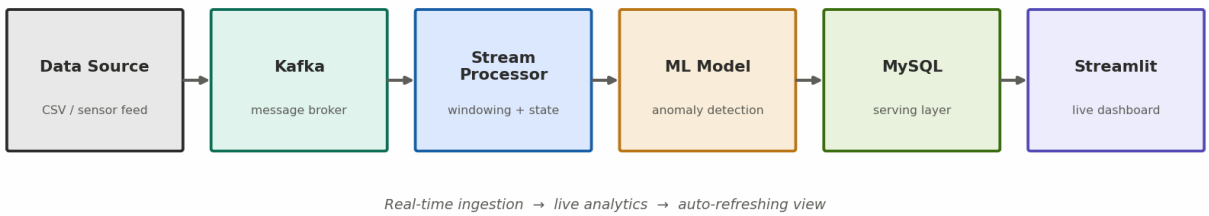


Figure 1. The six layers every streaming project follows. Data flows left to right.

The single most important idea in this whole project: **streaming ingestion does not automatically make your dashboard live.** A dashboard is only real-time if it (a) reads fresh data and (b) refreshes on its own. Many students build a streaming backend and then wonder why the dashboard looks frozen — Section 8 explains exactly why and how to fix it.

2. What Your Mini-Project Must Contain

Your project must implement all six layers below. Three of them are mandatory in a stronger sense: you must use at least one tool from each of the highlighted categories. Everything else is your choice.

#	Layer	What it does	Tool options (choose one)	Required
1	Data source	The event stream	CSV/JSONL replay, IoT feed, API, logs	Any source
2	Streaming tool	Transports the stream	Kafka, Storm, Spark, Flink	YES — one

3	Stream processor	Computes live metrics	Python, Spark, Flink, Kafka Streams	Required
4	ML algorithm	Scores each event	IsolationForest, SVM, K-Means, ARIMA	YES — one
5	Database	Stores state + history	MySQL, PostgreSQL, MongoDB, InfluxDB	Any store
6	Visualization	Live auto-refresh view	Streamlit, Dash, Grafana, Power BI	YES — one

Minimum to pass: at least one streaming tool (Kafka / Storm / Spark / Flink), at least one machine-learning algorithm, and at least one visualization tool — all wired together so the pipeline runs end to end and the dashboard updates live.

Deliverables: (1) working code for all layers, (2) a short report like this one describing your design and results, and (3) a live demonstration or screen recording showing the dashboard updating as data streams.

3. The Worked Example: Smart-Meter Data

To make each step concrete, we build one full project together. The dataset is per-minute household electricity readings. Each row is one minute:

```
Date_Time, Global_active_power, Global_reactive_power,
Global_intensity, Sub_metering_1, Sub_metering_2,
Sub_metering_3
2006-12-16 17:24:00, 4.216, 0.418, 18.4, 0.0, 1.0, 17.0
2006-12-16 17:25:00, 5.360, 0.436, 23.0, 0.0, 1.0, 16.0
```

The star column is Global_active_power (household power draw in kW). Because a new reading arrives every minute, replaying this file slowly makes a convincing live stream — you watch the power curve draw itself.

Our goal: stream the readings, compute a live rolling average and running peak, use a machine-learning model to flag unusual power spikes, store everything in a database, and show it all on a self-updating dashboard.

4. Layers 1 & 2: Streaming the Data with Kafka

Kafka is a **message broker**: a durable pipe that carries events from a *producer* (who sends) to a *consumer* (who processes). It keeps producer and consumer independent, so they can run at different speeds. We create a *topic* (a named channel) called smartmeter-events.

4.1 The producer

The producer reads the CSV one row at a time and sends each as a JSON message. The small delay between sends is what turns a static file into a live stream:

```
import csv, json, time
from kafka import KafkaProducer

producer = KafkaProducer(bootstrap_servers='127.0.0.1:9092',
```

```
    value_serializer=lambda v: json.dumps(v).encode('utf-8'))

with open(DATASET) as f:
    for row in csv.DictReader(f):
        event = {'reading_time': row['Date_Time'],
                 'global_active_power': float(row['Global_active_power'])}
        producer.send('smartmeter-events', value=event)
        time.sleep(0.2)   # 5 readings/sec -> feels live
```

Tip: real sensor data has gaps. Guard every number so a blank or ? doesn't crash the producer — convert bad values to None and skip rows with no reading.

5. Layer 3: The Stream Processor (Windowing)

The processor consumes each event and computes live metrics. This is where the core streaming concept lives: it keeps state in memory between events. The key idea is the sliding window — the last 30 readings — used to compute a rolling average.

```
from collections import deque
window = deque(maxlen=30)      # keeps only the last 30 readings
peak = 0.0

for message in consumer:
    power = message.value['global_active_power']
    window.append(power)
    rolling_avg = sum(window) / len(window)   # live 30-min average
    peak = max(peak, power)                   # running peak
    # ... write to database ...
```

The deque(maxlen=30) automatically drops the oldest reading when the 31st arrives, so the average always reflects the most recent 30 minutes. This three-line idea *is* windowed stream processing. The rolling average lags behind sudden spikes — that visible lag is the window doing its job.

6. Layer 4: The Machine-Learning Model

We add an **anomaly detector** — a model that flags unusual power readings in real time. We use IsolationForest (from scikit-learn). It is unsupervised, meaning it needs no labelled examples: it learns what 'normal' looks like and flags the rest.

6.1 The train / serve split

This is the professional pattern for ML in streaming systems, and an important concept to show:

1. **Train offline (once).** Fit the model on historical data and save it to a file. This is slow and happens before the stream runs.
2. **Serve live (always).** The processor loads the saved model once and scores each incoming reading in microseconds. The stream never re-trains.

6.2 Training the model

```
from sklearn.ensemble import IsolationForest
import joblib, pandas as pd
```

```
df = pd.read_csv(DATASET, usecols=['Global_active_power']).dropna()
model = IsolationForest(contamination=0.01, random_state=42)
model.fit(df[['Global_active_power']])
joblib.dump(model, 'anomaly_model.pkl')   # saved artifact
```

The key knob is contamination — the fraction of data you expect to be anomalous (here 1%). Raise it to flag more readings. This is a great parameter to experiment with and discuss in your report.

6.3 Scoring live in the processor

```
model = joblib.load('anomaly_model.pkl')   # load once at startup

for message in consumer:
    power = message.value['global_active_power']
    prediction = model.predict([[power]])[0]   # 1 = normal, -1 = anomaly
    is_anomaly = 1 if prediction == -1 else 0
    # ... store power, rolling_avg, is_anomaly ...
```

7. Layer 5: The Database (Serving Layer)

The database stores what the dashboard will read. This decouples processing from visualization — the processor writes, the dashboard reads, and neither waits for the other. We use two tables:

- meter_readings — every minute's raw values plus the rolling average and the anomaly flag. This feeds the live curve.
- meter_metrics — a single row, updated every minute, holding the latest snapshot: current power, rolling average, peak, total energy. This feeds the KPI tiles.

Why a single-row metrics table? Because 'current state' is one thing, not a growing list. Updating one row keeps it tiny and fast. The raw table grows, so in a longer project you would filter it by time or archive old rows.

```
CREATE TABLE meter_readings (
    reading_pk BIGINT AUTO_INCREMENT PRIMARY KEY,
    reading_time DATETIME,
    global_active_power DECIMAL(10,3),
    rolling_avg_30 DECIMAL(10,3),
    is_anomaly TINYINT DEFAULT 0
);
```

8. Layer 6: The Live Dashboard

This is where students most often get stuck, so read carefully. A dashboard feels frozen for two independent reasons, and you must fix both:

1. **No new data is arriving.** If all events are already processed, re-reading the database shows the same numbers. Fix: pace the producer so data trickles in.
2. **The view doesn't re-query.** If it only refreshes when you click, it won't update on its own. Fix: a timed auto-refresh.

We use Streamlit (a Python dashboard library). The dashboard opens a fresh database connection, reads the current state, draws the KPIs and the power curve, marks anomalies as separate points, and refreshes every 2 seconds:

```
import streamlit as st, pandas as pd, mysql.connector
st.markdown("<meta http-equiv='refresh' content='2'>",
            unsafe_allow_html=True)   # auto-refresh every 2s

conn = mysql.connector.connect(host='localhost', user='meter_app',
       password='meter_pass', database='smartmeter')
recent = pd.read_sql('SELECT reading_time, global_active_power,'
    ' rolling_avg_30, is_anomaly FROM meter_readings'
    ' ORDER BY reading_pk DESC LIMIT 120', conn)

st.line_chart(recent.set_index('reading_time')[
    ['global_active_power','rolling_avg_30']])
```

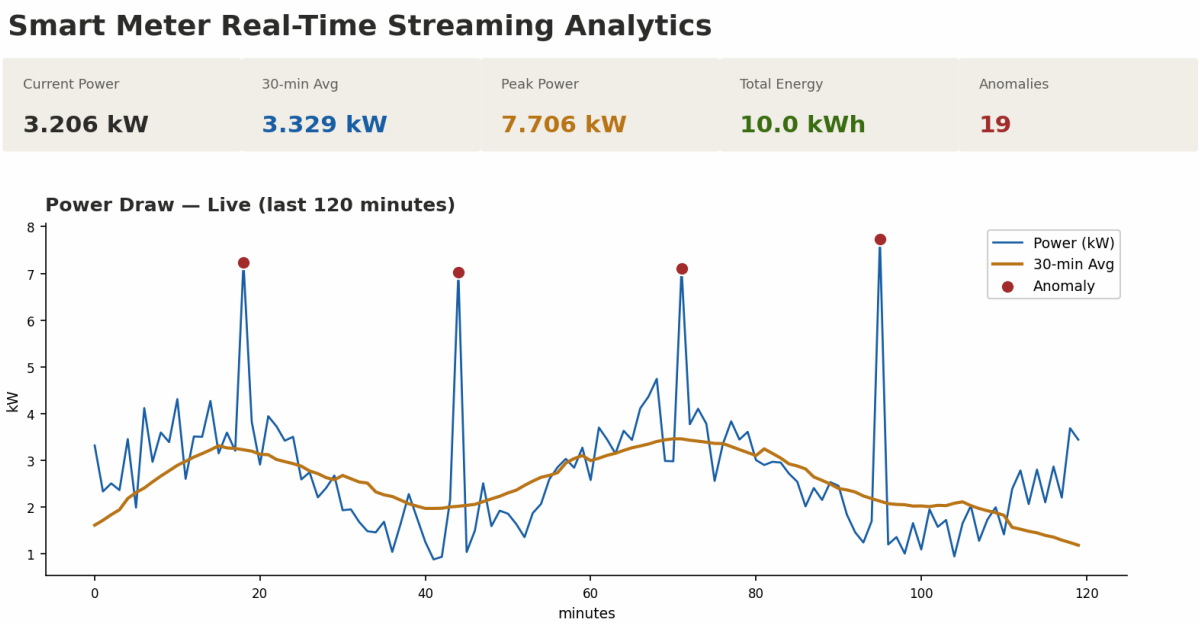


Figure 2. The live dashboard: KPI tiles, the power curve with its rolling average, and red markers where the ML model flagged anomalies.

Because the dashboard opens a new connection every refresh, it always shows fresh data — never a stale snapshot. The red dots are the ML model's anomalies appearing in real time as spikes stream in.

9. Running the Whole Pipeline

Start each part in its own terminal, in this order. Each keeps running.

1. Start ZooKeeper, then Kafka (the broker).
2. Create the topic (once).
3. Train the ML model (once) so the model file exists.
4. Start the stream processor (loads the model, writes to the database).

5. Open the live dashboard in your browser.
6. Start the producer — watch the dashboard come alive.

Verify data is flowing with a quick database query — the reading count should climb and the anomaly count should be non-zero once high spikes stream through.

10. Demonstrating It Live

For your presentation, this sequence makes the 'live' nature obvious:

- Reset the tables so counts start from zero.
- Start everything, open the dashboard, then start the paced producer.
- Watch the power curve extend, the rolling average glide, and the KPI tiles tick over on their own.
- Point out red anomaly markers appearing on the spikes — that is the ML model working live.

A powerful teaching moment: pause the producer. The dashboard keeps refreshing but the numbers freeze — proving that 'live' depends on data movement, not just the refresh timer. Resume, and the curve continues.

11. Writing Your Own Report

Your report should let a reader understand and rebuild your project. A good structure:

Section	What to include
Abstract	2–4 sentences: what you built and the main result
Architecture	A diagram of your six layers and the tools you chose
Dataset	Your data's format and what one event looks like
Each layer	A short section per layer with a code snippet
ML model	Which algorithm, how you trained it, what it detects
Dashboard	A screenshot and how it stays live
Results	Your numbers: totals, peak, anomaly count, screenshots
Conclusion	What you learned; ideas to extend it

Write plainly. Explain *why* each layer exists, not just what the code does. Show one screenshot of your live dashboard — it is the proof your pipeline works.

12. Ideas to Make Yours Stand Out

- Swap the tools: try Spark or Flink instead of a plain Python processor, or Grafana instead of Streamlit.
- Add a forecast: predict the next 30 minutes of power with ARIMA or Prophet and plot it ahead of the live curve.
- Add a time-window metric: energy used in the last hour, updating live.
- Try a different dataset: website clicks, stock ticks, weather sensors, or delivery orders — the six layers stay the same.
- Tune the model: compare contamination levels and discuss the trade-off between catching real anomalies and false alarms.

Appendix: Submission Checklist

- At least one streaming tool (Kafka / Storm / Spark / Flink) — working.
- At least one ML algorithm — trained and scoring live.
- At least one visualization tool — auto-refreshing on a live data source.
- A database serving layer between processor and dashboard.
- The full pipeline runs end to end and the dashboard updates on its own.
- A written report and a live demo or screen recording.